LANChat-H: Security Cheat Sheet

demo & viva

Layers (what, how, number to quote)

Layer	How it works	Quote
Transport	TLS 1.2+ on chat and file connections; self-signed cert, fingerprint **pinned on first connect** (like SSH); a changed fingerprint blocks the connection	Stops sniffing / man-in-the-middle after first connect
Passwords	scrypt, salt 16 B, N=2^{14}, r=8, p=1; min 6 chars, not a common password	One check ≈27 ms
Online guessing	Lockout: 5 fails → 30 s, doubling to 15 min; token-bucket rate limit; 240 connections/min and 10 signups/h per address	
Sessions	Saved sign-in token valid 12 h; server stores only its SHA-256	
Authorization	Roles admin / member / guest, checked on the *server* by `can(role, action)`	Client is never trusted
Frames & slow clients	4-byte length prefix, max 4096 B; bounded outbox (100) per client, slow client dropped; heartbeat 10 s / idle 30 s	Stalled receiver: 6000/6000 delivered vs relay stops at 232
Files	One-time expiring ticket, separate TLS data connection, SHA-256 checked, name sanitised (80 chars), executables blocked, quota, stored by hash	
Audit	Each record includes the hash of the previous one; head marker catches truncation; `chat.py admin audit -verify`	Editing history breaks the chain

End-to-end encryption (server stores only ciphertext)

- **Identity:** X25519 (key agreement) + Ed25519 (signatures), self-signed bundle. Primitives: ChaCha20-Poly1305, HKDF-SHA256, scrypt (N=2^{15}) (`cryptography` package).
- **Private message:** random message key encrypts text; key sealed for recipient *and* sender (ephemeral X25519 + HKDF); Ed25519 signature; sender/recipients bound into the authenticated data. Limits: 1500 B text, 3000-char envelope. Cost ≈0.3 ms.
- **Files:** 64 KiB chunks, nonce = chunk number + final flag (drop/reorder/truncate fails); key, real name and type are inside the encrypted message; 20 MB ≈31 ms.
- **Group room:** per-sender chain key per *epoch*; message n key = n one-way HMAC steps; chain sealed to each member as it stands, so **late joiners read only forward**; server enforces rising message numbers (jump ≤1000); removal or admin re-key starts a new epoch the removed person never receives.
- **Trust:** contact keys pinned on first sight; change blocks sending until accepted; **safety number** = 60 digits (5200 SHA-512 rounds), compare aloud.
- **Keys at rest:** key file wrapped by passphrase (scrypt); 24-char **recovery code** (≈119 bits) wraps an opaque server backup; **rotation** keeps old identities; **app lock** forgets keys after 15 min idle.

Live demo moves

`python3 chat.py host -name "Demo"`	Start the server
`python3 chat.py`	Client; first sign-in shows key set-up
`python3 demo/showcase.py`	Scripted rehearsal, no second computer
`Right-click name → Verify security code`	Show the 60 digits, mark verified
`python3 -c "print(open('accounts.db','rb').read().count(b'secret words'))"`	After sending those words privately: prints 0
`Admin console → Settings → Encrypt the group room`	Turn on the room; disable a member to show the new epoch
`python3 chat.py admin audit -verify`	Audit chain intact
`python3 -m unittest`	About 450 tests pass

If asked

Can the host read private chats?	No: ciphertext and public keys only; the wire-scan test checks every frame, the database and stored files.
What stops a key swap?	Pinning plus safety numbers; a change blocks sending.
What does the server still see?	Who talks to whom, when, sizes, mentions, roles, room membership.
Forward secrecy?	No. A stolen long-term key exposes what that person could read; a ratchet is future work.
Lost passphrase and recovery code?	Old encrypted messages are gone by design.
Biggest caveats	Trust on first use; one machine measured; no external security review; no asyncio comparison.